

Python Functional Toolkit

lambda • map() • filter() • reduce()

λ

$\Rightarrow$

$\oplus$

$\emptyset$

Σ

A quick visual reference guide with runnable examples and expected output

λ Lambda Functions

A **lambda function** is a small, anonymous (unnamed) function in Python. It's written in a single line and is ideal for short, throwaway logic — especially when used alongside `map()`, `filter()`, and `reduce()`.

def	lambda
Has a function name	Anonymous (can be assigned to a variable)
Can contain multiple statements	Only one expression
Uses return	Returns the expression automatically
Better for complex logic	Best for short, simple functions

Syntax → `lambda arguments: expression`

Example 1 — def vs lambda (Add two numbers)

```
def add(x, y):  
    return x + y  
print(add(3, 5))           # using def  
  
add = lambda x, y: x + y  
print(add(3, 5))           # using lambda
```

Output: 8 | 8

Example 2 — Square & Double a number

```
square = lambda x: x ** 2  
print(square(4))  
  
double = lambda x: x * 2  
print(double(10))
```

Output: 16 | 20

Example 3 — lambda combined with map(), filter(), reduce()

```
numbers = [1, 2, 3, 4, 5]  
  
result = map(lambda x: x * 2, numbers)  
print(list(result))           # map  
  
result = filter(lambda x: x % 2 == 0, numbers)  
print(list(result))           # filter  
  
from functools import reduce  
result = reduce(lambda x, y: x + y, numbers)  
print(result)                 # reduce
```

Output: [2, 4, 6, 8, 10] | [2, 4] | 15

💡 **Tip:** Use lambda for quick, one-line logic. If your logic needs multiple steps or is reused often, define a normal function with `def` instead.

📄 Practice Questions — lambda (20)

1 Write a lambda that adds 10 to a number.

`add_ten = lambda x: x + 10` → `add_ten(5)` **15**

2 Write a lambda that checks if a number is even.

```
is_even = lambda x: x % 2 == 0 → is_even(4) True
```

3 Write a lambda that returns the square of a number.

```
square = lambda x: x ** 2 → square(6) 36
```

4 Write a lambda that concatenates two strings.

```
concat = lambda a, b: a + b → concat("Hello ", "World") 'Hello World'
```

5 Write a lambda that returns the larger of two numbers.

```
max_num = lambda a, b: a if a > b else b → max_num(4, 9) 9
```

6 Write a lambda to convert Celsius to Fahrenheit.

```
c_to_f = lambda c: (c * 9/5) + 32 → c_to_f(0) 32.0
```

7 Write a lambda that returns the cube of a number.

```
cube = lambda x: x ** 3 → cube(3) 27
```

8 Write a lambda that checks if a string starts with a vowel.

```
starts_vowel = lambda s: s[0].lower() in "aeiou" → starts_vowel("Apple") True
```

9 Write a lambda that returns the average of two numbers.

```
avg = lambda a, b: (a + b) / 2 → avg(4, 8) 6.0
```

10 Write a lambda that reverses a string.

```
reverse = lambda s: s[::-1] → reverse("hello") 'olleh'
```

11 Write a lambda that returns the length of a string.

```
length = lambda s: len(s) → length("python") 6
```

12 Write a lambda that converts a string to uppercase.

```
to_upper = lambda s: s.upper() → to_upper("hi") 'HI'
```

13 Write a lambda that adds three numbers.

```
add3 = lambda a, b, c: a + b + c → add3(1, 2, 3) 6
```

14 Write a lambda that checks divisibility by 3.

```
div3 = lambda x: x % 3 == 0 → div3(9) True
```

15 Write a lambda that returns the absolute difference of two numbers.

```
diff = lambda a, b: abs(a - b) → diff(10, 4) 6
```

16 Write a lambda that checks if a number is negative.

```
is_neg = lambda x: x < 0 → is_neg(-5) True
```

17 Use a lambda with sorted() to sort tuples by their second element.

```
data = [("a",3),("b",1),("c",2)]; sorted(data, key=lambda x: x[1]) → [('b',1), ('c',2), ('a',3)]
```

18 Use a lambda with sorted() to sort strings by length.

```
words=["banana","kiwi","fig","apple"]; sorted(words, key=lambda w: len(w)) → ['fig','kiwi','apple','banana']
```

19 Write a lambda that checks if a string is a palindrome.

```
is_pal = lambda s: s == s[::-1] → is_pal("madam") True
```

20 Use a lambda with map() to double each number in a list.

```
list(map(lambda x: x * 2, [3, 6, 9])) → [6, 12, 18]
```

⇒ map() Function

`map()` applies a given function to **every item** in an iterable (list, tuple, set, etc.) and returns a **map object** (an iterator) — great for transforming data.

Function	Purpose
<code>map(function, iterable)</code>	Apply a function to every element of an iterable
Return value	A map object (iterator)
Common use	Transform data — multiply numbers, convert types, change text case

Example 1 — Multiply each element by 2

```
numbers = [1, 2, 3, 4, 5]
result = map(lambda x: x * 2, numbers)
print(list(result))
```

Output: [2, 4, 6, 8, 10]

Example 2 — Convert strings to integers & names to uppercase

```
numbers = ["1", "2", "3", "4"]
print(list(map(int, numbers)))

names = ["ali", "sara", "john"]
print(list(map(str.upper, names)))
```

Output: [1, 2, 3, 4] ['ALI', 'SARA', 'JOHN']

Example 3 — Using a normal function

```
def square(x):
    return x ** 2

numbers = [1, 2, 3, 4, 5]
result = map(square, numbers)
print(list(result))
```

Output: [1, 4, 9, 16, 25]

Example 4 — map() returns an iterator, not a list

```
numbers = [1, 2, 3]
result = map(lambda x: x + 1, numbers)
print(result)           # <map object at 0x...>
print(list(result))     # [2, 3, 4]
```

Example 5 — Real-world: add 15% tax to prices

```
prices = [100, 200, 300, 400]
final_prices = list(map(lambda x: x * 1.15, prices))
print(final_prices)
```

Output: [115.0, 230.0, 345.0, 460.0]

💡 **Tip:** Always wrap `map()` in `list()` (or another container) to see or use the actual results — otherwise you just get an iterator object.

Practice Questions — map() (20)

1 Add 5 to each number in a list.

```
list(map(lambda x: x+5, [1,2,3])) → [6, 7, 8]
```

2 Convert a list of floats to integers.

```
list(map(int, [1.9, 2.5, 3.1])) → [1, 2, 3]
```

3 Get the length of each word in a list.

```
list(map(len, ["cat","apple","hi"])) → [3, 5, 2]
```

4 Square each number in a list.

```
list(map(lambda x: x**2, [1,2,3,4])) → [1, 4, 9, 16]
```

5 Convert a list of numbers to strings.

```
list(map(str, [1,2,3])) → ['1','2','3']
```

6 Capitalize each name in a list.

```
list(map(str.capitalize, ["ali","sara"])) → ['Ali','Sara']
```

7 Add two lists element-wise using map with two iterables.

```
list(map(lambda x,y: x+y, [1,2,3], [10,20,30])) → [11, 22, 33]
```

8 Reverse each string in a list.

```
list(map(lambda s: s[::-1], ["abc","xyz"])) → ['cba','zyx']
```

9 Convert a list of Celsius temperatures to Fahrenheit.

```
list(map(lambda c: (c*9/5)+32, [0,20,37])) → [32.0, 68.0, 98.6]
```

10 Check whether each number in a list is even.

```
list(map(lambda x: x%2==0, [1,2,3,4])) → [False, True, False, True]
```

11 Get the absolute value of each number in a list.

```
list(map(abs, [-3,4,-7])) → [3, 4, 7]
```

12 Strip whitespace from each string in a list.

```
list(map(str.strip, [" hi ", " bye "])) → ['hi','bye']
```

13 Compute the square root of each number (use math.sqrt).

```
import math; list(map(math.sqrt, [4,9,16])) → [2.0, 3.0, 4.0]
```

14 Cube each number in a list.

```
list(map(lambda x: x**3, [1,2,3])) → [1, 8, 27]
```

15 Convert a list of number-strings to floats.

```
list(map(float, ["1.5","2.3"])) → [1.5, 2.3]
```

16 Triple each number using a normal function.

```
def triple(x): return x*3 list(map(triple, [1,2,3])) → [3, 6, 9]
```

17 Swap the case of each string in a list.

```
list(map(str.swapcase, ["Hello","WORLD"])) → ['hELLO','world']
```

18 Use map() on a tuple to double each value.

```
tuple(map(lambda x: x*2, (1,2,3))) → (2, 4, 6)
```

19 Get the length of each sublist in a list of lists.

```
list(map(len, [[1,2],[3,4,5],[6]])) → [2, 3, 1]
```

20 Add 8% tax to a list of prices.

```
list(map(lambda p: p*1.08, [100,200])) → [108.0, 216.0]
```

🔗 filter() Function

`filter()` selects only the elements of an iterable that satisfy a **condition** (a function that returns True or False) — useful for keeping only the data you need.

Syntax → `filter(function, iterable)`

Example 1 — Keep only even numbers

```
numbers = [1, 2, 3, 4, 5, 6]
result = filter(lambda x: x % 2 == 0, numbers)
print(list(result))
```

Output: [2, 4, 6]

Example 2 — Keep numbers greater than 10

```
numbers = [5, 12, 8, 20, 15]
result = filter(lambda x: x > 10, numbers)
print(list(result))
```

Output: [12, 20, 15]

Example 3 — Keep names that start with "A"

```
names = ["Ali", "Sara", "Ahmed", "John", "Adam"]
result = filter(lambda name: name.startswith("A"), names)
print(list(result))
```

Output: ['Ali', 'Ahmed', 'Adam']

Example 4 — Using a normal function

```
def is_even(x):
    return x % 2 == 0

numbers = [1, 2, 3, 4, 5, 6]
result = filter(is_even, numbers)
print(list(result))
```

Output: [2, 4, 6]

Example 5 — Real-world: students who passed (score ≥ 50)

```
scores = [45, 70, 30, 90, 60, 40]
passed = list(filter(lambda score: score >= 50, scores))
print(passed)
```

Output: [70, 90, 60]

💡 **Tip:** Unlike `map()`, the function passed to `filter()` must always return a boolean — elements where it's True are kept.

📖 Practice Questions — filter() (20)

1 Filter out only the odd numbers.

```
list(filter(lambda x: x%2!=0, [1,2,3,4,5])) → [1, 3, 5]
```

2 Filter numbers divisible by 5.

```
list(filter(lambda x: x%5==0, [5,7,10,12,15])) → [5, 10, 15]
```

3 Filter out negative numbers.

```
list(filter(lambda x: x<0, [-2,3,-5,7])) → [-2, -5]
```

4 Filter strings longer than 3 characters.

```
list(filter(lambda s: len(s)>3, ["hi","hello","cat","world"])) → ['hello','world']
```

5 Filter names ending with the letter "a".

```
list(filter(lambda n: n.endswith('a'), ["Lina","Omar","Layla"])) → ['Lina','Layla']
```

6 Filter numbers between 10 and 50 (inclusive).

```
list(filter(lambda x: 10<=x<=50, [5,20,45,60])) → [20, 45]
```

7 Filter out empty strings from a list.

```
list(filter(None, ["hi","", "world",""])) → ['hi','world']
```

8 Filter palindromic words from a list.

```
list(filter(lambda w: w==w[::-1], ["level","hello","madam"])) → ['level','madam']
```

9 Filter prime numbers using a normal function.

```
def is_prime(n): return n>1 and all(n%i for i in range(2,n)) list(filter(is_prime, [2,3,4,5,6,7])) → [2, 3, 5, 7]
```

10 Filter vowels out of a list of letters.

```
list(filter(lambda c: c in 'aeiou', ['a','b','c','e','i'])) → ['a','e','i']
```

11 Filter perfect squares from a list.

```
list(filter(lambda x: int(x**0.5)**2==x, [1,2,3,4,9,10,16])) → [1, 4, 9, 16]
```

12 Remove all None values from a list.

```
list(filter(lambda x: x is not None, [1,None,2,None,3])) → [1, 2, 3]
```

13 Filter strings that contain at least one digit.

```
list(filter(lambda s: any(c.isdigit() for c in s), ["abc","alb","123"])) → ['a1b','123']
```

14 Filter students who failed (score below 50).

```
list(filter(lambda s: s<50, [45,70,30,90,60,40])) → [45, 30, 40]
```

15 Filter numbers from 1-15 that are multiples of 3 or 5.

```
list(filter(lambda x: x%3==0 or x%5==0, range(1,16))) → [3,5,6,9,10,12,15]
```

16 Filter words that start with a vowel.

```
list(filter(lambda w: w[0].lower() in 'aeiou', ["Apple","Banana","Egg"])) → ['Apple','Egg']
```

17 Filter numbers greater than the list's average.

```
nums=[1,2,3,4,5]; avg=sum(nums)/len(nums) list(filter(lambda x: x>avg, nums)) → [4, 5]
```

18 Filter only the "truthy" values from a mixed list.

```
list(filter(bool, [0,1,'',None,'hi',False,5])) → [1, 'hi', 5]
```

19 Filter words that are exactly 4 letters long.

```
list(filter(lambda w: len(w)==4, ["cat","tree","frog","sun"])) → ['tree','frog']
```

20 Filter numbers from 1-30 divisible by both 2 and 3.

```
list(filter(lambda x: x%2==0 and x%3==0, range(1,31))) → [6,12,18,24,30]
```


Σ reduce() Function

`reduce()` combines all elements of an iterable into a **single value** by repeatedly applying a function to pairs of elements, until only one result remains. It lives in the `functools` module.

Syntax →

```
from functools import reduce
reduce(function, iterable)
```

Example 1 — Sum all numbers

```
from functools import reduce
numbers = [1, 2, 3, 4, 5]
result = reduce(lambda x, y: x + y, numbers)
print(result)
```

Output: 15

Example 2 — Multiply all numbers

```
numbers = [1, 2, 3, 4]
result = reduce(lambda x, y: x * y, numbers)
print(result)
```

Output: 24

Example 3 — Find the largest number

```
numbers = [8, 3, 12, 5, 9]
largest = reduce(lambda x, y: x if x > y else y, numbers)
print(largest)
```

Output: 12

Example 4 — Real-world: total shopping bill

```
prices = [120, 250, 80, 150]
total = reduce(lambda x, y: x + y, prices)
print(total)
```

Output: 600

💡 **Tip:** `reduce()` always needs an import from `functools` — unlike `map()` and `filter()`, which are built-in.

Practice Questions — reduce() (20)

1 Find the sum of a list of numbers.

```
reduce(lambda a,b: a+b, [1,2,3,4]) → 10
```

2 Find the product of a list of numbers.

```
reduce(lambda a,b: a*b, [1,2,3,4]) → 24
```

3 Find the maximum value in a list.

```
reduce(lambda a,b: a if a>b else b, [3,7,2,9,4]) → 9
```

4 Find the minimum value in a list.

```
reduce(lambda a,b: a if a< b else b, [3,7,2,9,4]) → 2
```

5 Concatenate a list of strings into one.

```
reduce(lambda a,b: a+b, ["Py","th","on"]) → 'Python'
```

6 Use reduce() with an initial value of 100.

```
reduce(lambda a,b: a+b, [1,2,3], 100) → 106
```

7 Count the total characters across a list of words.

```
reduce(lambda a,b: a+len(b), ["cat","dog","bird"], 0) → 10
```

8 Find the longest string in a list.

```
reduce(lambda a,b: a if len(a)>len(b) else b, ["hi","hello","hey"]) → 'hello'
```

9 Flatten a list of lists into one list.

```
reduce(lambda a,b: a+b, [[1,2],[3,4],[5]]) → [1, 2, 3, 4, 5]
```

10 Join a list of letters into a comma-separated string.

```
reduce(lambda a,b: a+", "+b, ["a","b","c"]) → 'a,b,c'
```

11 Find the total of a list of vote counts.

```
reduce(lambda a,b: a+b, [120,340,90]) → 550
```

12 Compute 5! (factorial) using reduce().

```
reduce(lambda a,b: a*b, range(1,6)) → 120
```

13 Find the sum of squares of a list.

```
reduce(lambda a,b: a+b**2, [1,2,3], 0) → 14
```

14 Compute the XOR of a list of numbers.

```
reduce(lambda a,b: a^b, [5,3,9]) → 15
```

15 Count the total number of words across sentences.

```
reduce(lambda a,b: a+len(b.split()), ["hi there","how are you"], 0) → 5
```

16 Find the total quantity from a list of (item, qty) tuples.

```
items=[("apple",3),("banana",2)] reduce(lambda a,b: a+b[1], items, 0) → 5
```

17 Find the shortest word in a list.

```
reduce(lambda a,b: a if len(a)<len(b) else b, ["fig","apple","banana"]) → 'fig'
```

18 Count the total number of vowels across a list of words.

```
reduce(lambda a,b: a+sum(1 for c in b if c in 'aeiou'), ["apple","orange"], 0) → 5
```

19 Find the union of a list of sets.

```
reduce(lambda a,b: a|b, [{1,2},{2,3},{3,4}]) → {1, 2, 3, 4}
```

20 Find the total sum of all numbers in a list of sublists.

```
reduce(lambda a,b: a+sum(b), [[1,2],[3,4],[5]], 0) → 15
```

Quick Recap

- **lambda** — write small anonymous functions inline
- **map()** — transform every element of an iterable
- **filter()** — keep only elements that satisfy a condition
- **reduce()** — combine all elements into one final value